RideSafety Auth Service - Learning & Integration Guide

Welcome to the **Auth Service** codebase! This microservice provides production-grade **Email + OTP** authentication, integrated with the Ory ecosystem (**Kratos, Hydra, Keto**) following **Clean Architecture** principles.

This guide details the system design, code flow, and the exact sequence of commands to run, test, and interact with the service.

---

1. Directory Structure

The service is structured as follows to ensure high separation of concerns:

```
auth/
├── cmd/
│   ├── auth-service/     # Main entry point (main.go)
│   └── migrate/          # Custom DB migrations runner tool
├── docs/                 # Documentation (like this guide)
├── migrations/           # PostgreSQL DB migrations (up/down SQL scripts)
└── internal/             # Private application source code
    ├── api/              # API layer: handlers, routes, request/response validation, mi
    ├── application/      # Application layer: use cases (register, login, logout, refre
    ├── domain/           # Domain layer: pure entities, repository interfaces, domain e
    ├── infrastructure/   # Infrastructure layer: postgres/redis/kafka/ory client adapte
    ├── platform/         # Cross-cutting adapters (cache, eventbus, identity orchestrat
    ├── shared/           # Shared utilities (OTP generator, app errors, response helper
    └── bootstrap/        # Server bootstrapper, configuration, DI container wiring

The external infrastructure orchestration configurations (PostgreSQL, Redis, Kafka, Ory
`configs/` and `deployments/compose/` inside the `auth` directory.
```

---

2. Core Flows (Architecture in Action)

Registration Flow

1. **POST** `/register` :

- The user submits their email, displayName, and deviceId.
- The system checks **Keto** to ensure the device is not already owned.
- Generates a 6-digit OTP, stores a hashed version in Postgres (`pending_auth`), and saves the registration details in `metadata` .
- Transactionally persists a `UserRegistered` or `OTPRequested` event into `outbox_events` and commits.
- The **Outbox Publisher** background worker picks up the event and streams it to Kafka on `auth.email.send` .

2. **POST** `/register/verify` :
   - The client submits the `requestId` and `otp` .
   - The system validates the OTP. If valid, it:
     - Creates the identity in **Ory Kratos**.
     - Creates the user in local Postgres (`users` table).
     - Deletes the `pending_auth` session.
     - Calls **Ory Hydra** to generate a token pair.
     - Returns the `accessToken` in the JSON body and the `refreshToken` in a secure `HttpOnly` cookie.

Login Flow

1. **POST** `/login` :
   - The user submits their email.
   - The system verifies the user exists in local Postgres.
   - Generates an OTP, hashes it, saves it to `pending_auth` , and transactionally writes an outbox event.
   - The worker publishes a `LOGIN_OTP` send event to Kafka.
2. **POST** `/login/verify` :
   - Client sends `requestId` and `otp` .
   - If verified, the system calls **Hydra** to generate new tokens, deletes the session, and returns the tokens.

---

3. Sequence of Commands (Step-by-Step)

Follow this sequence of commands in your terminal to set up, build, migrate, and test the auth service locally.

Step 1: Initialize Dependencies & Build Project Root

Verify that your Go environment is correct:

```
# Check Go version (requires Go 1.26+)
go version

# Download Go dependencies
go mod tidy
```

Step 2: Spin Up the Ory & Infrastructure Stack

Start PostgreSQL, Redis, Kafka, Ory Kratos, Ory Hydra, Ory Keto, Mailpit, and Kafka-UI:

```
# Start Docker Compose services in the background
docker-compose -f deployments/compose/docker-compose.yml up -d
```

You can view the health status of all containers:

```
docker-compose -f deployments/compose/docker-compose.yml ps
```

Step 3: Run Database Migrations

Run the database migrations up command to create the tables (`users` , `pending_auth` , `outbox_events` , `audit_logs` , `idempotency_keys`):

```
# Compile and run migrations
go run cmd/migrate/main.go up
```

Step 4: Seed Ory Hydra client

For the Auth Service to request tokens from Hydra, an OAuth2 client must be registered in Hydra:

```
# Create the client 'auth-service-client' in Ory Hydra
docker-compose -f deployments/compose/docker-compose.yml exec hydra \
  hydra clients create \
  --endpoint http://localhost:4445/ \
  --id auth-service-client \
  --secret auth-service-secret \
  --grant-types authorization_code,refresh_token \
  --response-types code \
  --scope openid,offline_access \
  --callbacks http://localhost:8080/api/v1/auth/callback
```

Step 5: Start the Auth Service

Now run the Go application locally:

```
# Run the Go auth service
go run cmd/auth-service/main.go
```

The HTTP server will start on port `8080` (endpoints base: `/api/v1/auth`).

---

4. How to Test the API (Request Sequencing)

1. Register User

```
curl -X POST http://localhost:8080/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -H "X-Correlation-ID: test-correlation-id" \
  -d '{
    "email": "learning@example.com",
    "deviceId": "DEVICE-100",
    "displayName": "Alex"
  }'
```

Response: Returns `202 Accepted` with a `requestId` and `expiresIn`.

```
{
  "success": true,
  "correlation_id": "test-correlation-id",
  "data": {
    "requestId": "550e8400-e29b-41d4-a716-446655440000",
    "expiresIn": 300
  }
}
```

2. Retrieve the OTP Code

Open your web browser and navigate to:

- **Mailpit Web UI**: `http://localhost:8025`

You will see an email containing the 6-digit OTP code (e.g., `584920`).

3. Verify Registration OTP

```
curl -X POST http://localhost:8080/api/v1/auth/register/verify \
  -H "Content-Type: application/json" \
  -H "X-Correlation-ID: test-correlation-id" \
  -d '{
    "requestId": "550e8400-e29b-41d4-a716-446655440000",
    "otp": "584920"
  }'
```

Response: Returns `200 OK` with `accessToken`, and sets the `refresh_token` in a cookie.

4. Refresh Token Pair

```
curl -X POST http://localhost:8080/api/v1/auth/refresh \
  -H "X-Correlation-ID: test-correlation-id" \
  --cookie "refresh_token=YOUR_REFRESH_TOKEN_HERE"
```

5. Start Login Flow (Subsequent Logins)

```
curl -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -H "X-Correlation-ID: test-correlation-id" \
  -d '{
    "email": "learning@example.com"
  }'
```

Retrieve the new OTP from Mailpit and verify:

```
curl -X POST http://localhost:8080/api/v1/auth/login/verify \
  -H "Content-Type: application/json" \
  -H "X-Correlation-ID: test-correlation-id" \
  -d '{
    "requestId": "NEW_REQUEST_ID",
    "otp": "NEW_OTP"
  }'
```

6. Logout

```
curl -X POST http://localhost:8080/api/v1/auth/logout \
  -H "X-User-Id: USER_KRATOS_ID" \
  -H "X-Correlation-ID: test-correlation-id" \
  --cookie "refresh_token=YOUR_REFRESH_TOKEN_HERE"
```

Response: `204 No Content` (clears cookie and revokes token).